A

Major Project

On

Customer Support Dataset With Multi-Intent Annotations for Conversational AI

(Submitted in partial fulfilment of the requirements of the award of Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

G. Shivani (227R1A1286)

N. Thriveni (227R1A12B7)

D. Akhilesh Reddy (227R1A1282)

Under the Guidance of

Mrs. K. Supriya Suhasini

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)

Recognized Under Section 2(f) & 12(B) of the UGCAct.1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

May, 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled "**Customer Support Dataset With Multi-Intent Annotations for Conversational AI**" being submitted by **G Shivani (227R1A1286), N.Thriveni (227R1A12B7) & D Akhilesh Reddy (227R1A1282)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mrs. K. Supriya Suhasini
Assistant Professor **Head of the Department**
INTERNAL GUIDE

Dr. A. Raji Reddy **Signature of External Examiner**

DIRECTOR

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mrs. K. Supriya Suhasini**, Assistant Professor for her exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Dr. J. Narasimharao, Mrs. K. Supriya Suhasini, Mr. D. Krishna Kumar, Mr. G. Shankar, Mrs. M. Siva Jyothi and Ms. G. Menaka** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our **Head of the Department**, for encouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus,** whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

G. Shivani (227R1A1286)
N. Thriveni (227R1A12B7)
D. Akhilesh Reddy (227R1A1282)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.
2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.
3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

DEPARTMENT MISSION:

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problem.

2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.

3. To enhance the leadership qualities and team spirit with social values to serve the society

ABSTRACT

This The global Conversational Artificial Intelligence (AI) market is experiencing rapid growth and is projected to reach nearly USD 32 billion by the year 2030. With the increasing adoption of digital platforms and automation in businesses, it is estimated that more than 80% of customer interactions will be handled by chatbots and virtual assistants in the near future. These AI-driven systems are capable of providing instant responses, reducing operational costs, and improving the overall customer experience. However, one of the major challenges in developing effective chatbot systems is the manual process of intent annotation and classification of customer queries. This process requires human effort to label each query with its appropriate intent and category, which is not only time-consuming but also prone to inconsistencies due to subjective interpretation. As the volume of customer queries grows, this manual approach becomes difficult to scale, ultimately limiting the efficiency of customer support systems.To address these limitations, this research proposes an advanced Natural Language Processing (NLP) based framework designed to improve the automation and accuracy of intent detection in conversational systems. Unlike traditional datasets that assume a single intent per query, this dataset supports multi-intent annotation, allowing the model to understand complex customer queries that contain more than one request or concern.The methodology begins with NLP preprocessing and Exploratory Data Analysis (EDA). During preprocessing, the raw textual data is cleaned and standardized through techniques such as tokenization, normalization, removal of stop words, and text transformation, which help convert unstructured text into a format suitable for machine learning models. Following this step, EDA is performed to analyze the dataset distribution, identify patterns, detect class imbalance, and visualize relationships between different intents and categories. This step helps in understanding the structure and quality of the dataset before applying predictive models.For feature extraction, the proposed framework employs Miniature Language Model (MiniLM), a lightweight yet powerful transformer-based embedding model. MiniLM captures the semantic meaning and contextual relationships between words, generating dense vector representations of the text.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of basic Intent and Category Application	**11**
Figure 3.3.1	Class Diagram of Customer support dataset With Multi-Intent Annotations For Conversatioal AI	**13**
Figure 3.3.2	Use Case Diagram of Customer support dataset With Multi-Intent Annotations For Conversatioal AI	**14**
Figure 3.3.3	Sequence Diagram of Customer support dataset With Multi-Intent Annotations For Conversatioal AI	**15**
Figure 3.3.4	Activity Diagram of Customer support dataset With Multi-Intent Annotations For Conversatioal AI	**16**

LIST OF SCREENSHOTS

SCREENSHOT NO	SCREENSHOT NAME	PAGE N0
Screenshot 5.1	Screenshot of Implementation Description	31
Screenshot 5.2	Screenshot of Dataset description	32
Screenshot 5.3	Screenshot of Results and Discussion	32
Screenshot 5.4	Screenshot of Exploratory Data Analysis	33
Screenshot 5.5	Screenshot of Target Column : Intent Classification	33
Screenshot 5.6	Screenshot of Receiver Operating Characteristic (ROC	34
Screenshot 5.7	Screenshot of Category Column : Intent Classification	35

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Uploading Dataset	37
Table 6.2.2	Classification	37

TABLE OF CONTENTS

ABSTRACT i
LIST OF FIGURES ii
LIST OF TABLES iv
1 INTRODUCTION **1**
1.1 PROJECT PURPOSE 1
1.2 PROJECT FEATURES 2
2 LITERATURE SURVEY **3**
2.1 REVIEW OF RELATED WORK 5
2.2 DEFINITION OF PROBLEM STATEMENT 7
2.3 EXISTING SYSTEM 7
2.4 PROPOSED SYSTEM 9
2.5 OBJECTIVES 9
2.6 HARDWARE & SOFTWARE REQUIREMENTS 10
2.6.1 HARDWARE REQUIREMENTS 10
2.6.2 SOFTWARE REQUIREMENTS 10
3 SYSTEM ARCHITECTURE & DESIGN **11**
3.1 PROJECT ARCHITECTURE 11
3.2 DESCRIPTION 12
3.3 UML DIAGRAM 12
3.3.1 CLASS DIAGRAM 13
3.3.2 USECASE DIAGRAM 14
3.3.3 SEQUENCE DIAGRAM 15
3.3.4 ACTIVITY DIAGRAM 16
4 IMPLEMENTATION **17**
4.1 ALGORITHMS USED 17
4.2 SAMPLE CODE 19
5 RESULTS & DISCUSSION **31**
6 VALIDATION **36**
6.1 INTRODUCTION 36
6.2 TEST CASES 37
6.2.1 UPLOADING DATASET 37
6.2.2 FEATURE EXTRACTION 37
6.2.3 CLASSIFICATION 38
7 CONCLUSION & FUTURE ASPECTS **39**
7.1 PROJECT CONCLUSION 39
7.2 FUTURE ASPECTS 39
8 BIBLIOGRAPHY **40**
8.1 REFERENCES 40
8.2 GITHUB LINK 41

1. INTRODUCTION

1. INTRODUCTION

The project, titled "Customer Support Dataset with Multi-Intent Annotations for Conversational AI," focuses on developing an intelligent AI-driven system capable of automatically understanding and classifying customer queries in customer support environments. The system is designed to identify multiple intents and categories within a single user query, enabling chatbots to understand complex customer requests more effectively. To achieve this, the project utilizes advanced Natural Language Processing (NLP) techniques, where Miniature Language Model (MiniLM) is used for extracting context-rich text embeddings, and a combination of Deep Neural Network (DNN) feature selection with K- Nearest Neighbors (KNN) classification is used for accurate prediction of intents and categories.

In modern digital services, organizations receive a large volume of customer queries through chat, email, and support platforms, making manual classification and response generation inefficient and time-consuming. Traditional chatbot systems often struggle to understand queries that contain multiple intents or contextual meanings, which reduces the quality of automated responses.

1.1 PROJECT PURPOSE

The primary purpose of this project is to improve the efficiency of customer support systems by automating the detection and classification of customer queries using Artificial Intelligence and Natural Language Processing (NLP). In many organizations, customer queries are still manually analyzed and categorized, which is time-consuming, inconsistent, and difficult to scale when the number of queries increases.

With the rapid growth of online services, businesses receive a large volume of customer requests through chatbots, emails, and support portals. Traditional chatbot systems often fail to understand queries that contain **multiple intents or contextual meanings**, which leads to inaccurate responses and reduced customer satisfaction. This project addresses these challenges by developing an **AI-powered conversational system** capable of identifying multiple intents within a single query.

1.2 PROJECT FEATURES

This project incorporates several key features to improve the accuracy and efficiency of customer query understanding and automated response generation in conversational AI systems:

Automated Intent Detection:

The system uses advanced Natural Language Processing (NLP) techniques to automatically analyze and understand customer queries. By using Miniature Language Model (MiniLM) embeddings, the system converts textual queries into meaningful vector representations that capture the context and semantic meaning of the text. This enables the model to accurately identify the intent behind customer queries and improve chatbot understanding.

Multi-Intent and Category Classification:

Unlike traditional chatbot systems that detect only a single intent, this project supports multi-intent classification, allowing the system to identify multiple intents present within a single customer query. Additionally, it classifies queries into different support categories, enabling more precise routing and response generation. This improves the chatbot's ability to handle complex queries and enhances overall customer service efficiency.

Balanced and Efficient Model Training:

Customer support datasets often contain imbalanced data, where some intents appear more frequently than others. To address this, the project applies the Synthetic Minority Over-sampling Technique (SMOTE) to generate synthetic samples for minority classes, ensuring balanced training data.

Scalable Chatbot Integration:

The trained model is integrated with a chatbot interface capable of processing real-time customer queries. This allows the system to automatically detect user intent and category and generate appropriate responses instantl

2. LITERATURE SURVEY

2. LITERATURE SURVEY

With the rapid growth of digital services and online platforms, customer support systems have become a crucial component of modern businesses. Organizations increasingly rely on conversational Artificial Intelligence (AI) to automate responses, handle customer queries efficiently, and improve user experience. However, traditional customer support systems struggle to accurately understand complex user queries, particularly when a single query contains multiple intents. This limitation has led to the development of advanced Natural Language Processing (NLP) techniques capable of detecting multiple intents within a single conversation.

This literature survey reviews previous research on conversational AI and intent detection systems used in customer support environments. The review focuses on traditional machine learning approaches, deep learning architectures, transformer-based language models, and hybrid models that integrate embedding techniques with classification algorithms. Special attention is given to methods that improve intent classification accuracy and enable multi-intent detection in conversational datasets.

Early research in conversational AI primarily relied on traditional machine learning algorithms for intent classification. Methods such as Naïve Bayes, Decision Trees, and Support Vector Machines were commonly used to classify customer queries based on manually engineered features. Researchers extracted features such as word frequency, TF-IDF vectors, and n-grams to train classification models. While these approaches were computationally efficient, they struggled to capture contextual relationships within sentences, often leading to inaccurate intent predictions.

Several studies explored the use of the K-Nearest Neighbours (KNN) algorithm for customer query classification. KNN is a simple yet effective algorithm that classifies queries based on similarity with previously labeled examples. Researchers demonstrated that KNN performs well when working with well-structured datasets and low-dimensional feature spaces. However, the model's performance decreases significantly when dealing with large datasets or high-dimensional text representations.

Another important development in conversational AI research is the introduction of multi-intent annotation datasets. Traditional intent detection systems assume that each query corresponds to a single intent. However, in real customer interactions, users often express multiple requests within a single message. Multi-intent datasets allow models to learn complex relationships between queries and multiple service categories.

Researchers have also explored integrating conversational AI systems with real-time chatbot interfaces. These systems enable automated responses to customer queries while continuously improving model performance through user interactions. Such implementations demonstrate the practical applications of NLP models in customer support environments.

In addition to intent detection, some studies have investigated the use of conversational context to improve response generation. Context-aware chatbots analyze previous conversation history to better understand user requirements. This approach enhances response relevance and improves overall user satisfaction.

Another emerging research direction involves the use of explainable AI techniques in conversational systems. Explainable AI methods allow developers to understand how models interpret user queries and make predictions. This transparency is essential for building trustworthy AI systems in customer support applications.

Despite significant progress in conversational AI research, several challenges remain. Traditional machine learning models fail to capture deep semantic relationships in text. Neural network models require large datasets and computational resources. Many intent detection systems still struggle with multi-intent queries and imbalanced datasets.

To address these challenges, recent studies propose integrating advanced NLP techniques with hybrid classification models. The combination of MiniLM embeddings, data balancing techniques, deep neural networks for feature selection, and KNN classification offers a promising solution for multi-intent detection in conversational datasets.

The literature survey highlights the evolution of conversational AI technologies from traditional machine learning algorithms to modern transformer-based models and hybrid classification frameworks. By leveraging contextual embeddings, balanced datasets, and advanced neural architectures, researchers have significantly improved the accuracy and scalability of intent detection systems.

2.1 REVIEW OF RELATED WORK

The development of conversational AI systems for customer support has gained significant attention in recent years due to the increasing demand for automated and efficient communication between businesses and users. Traditional customer support systems often rely on manual responses or rule-based chatbots, which struggle to understand complex user queries. Modern Natural Language Processing (NLP) techniques and machine learning models have enabled intelligent chatbots capable of understanding user intent and providing relevant responses. However, many real-world customer queries contain multiple intents, making accurate classification a challenging task. This literature survey reviews previous research on conversational AI systems, focusing on intent detection techniques, embedding models, and classification algorithms used in customer support datasets.

1 Traditional Content Moderation Approaches

Early conversational systems relied heavily on rule-based architectures and predefined scripts to interact with users. These systems used pattern matching and keyword-based rules to identify user requests and provide responses. While rule-based chatbots were simple to design and implement, they lacked flexibility and struggled to handle variations in user language. Furthermore, these systems were unable to understand complex queries containing multiple requests within a single message.

2.Machine Learning-Based Approaches

With the advancement of machine learning, researchers began using algorithms such as Support Vector Machines (SVM), Decision Trees, Naïve Bayes, and K-Nearest Neighbors (KNN) to classify customer queries. These models relied on feature extraction techniques like Bag-of-Words and TF-IDF to convert text data into numerical representations. Among these methods, KNN gained popularity due to its simplicity and effectiveness in similarity-based classification tasks. However, traditional machine learning models often struggle to capture contextual meaning and semantic relationships within text, which limits their performance in complex conversational datasets.

3.Deep Learning-Based Approaches

Deep learning techniques have significantly improved the ability of conversational AI systems to understand natural language. Deep Neural Networks (DNNs) allow models to automatically learn meaningful features from textual data without relying on manual feature engineering. These models can capture hidden patterns and relationships between words, enabling better intent detection and classification. Despite their advantages, deep learning models often require large datasets and substantial computational resources, which can make them difficult to deploy in real-time customer support systems.

4. Recent Advances: Attention Mechanisms & Transformer-Based Models

Recent advancements in Natural Language Processing have introduced embedding- based techniques that improve text representation for conversational AI systems. Word embedding models convert textual data into dense vector representations that capture semantic relationships between words. Transformer-based models such as MiniLM have further enhanced text understanding by generating contextual embeddings for entire sentences. These embeddings help conversational systems better interpret user queries and improve intent classification accuracy while maintaining computational efficiency.

This review highlights the evolution of conversational AI techniques used in customer support systems, emphasizing the transition from traditional rule-based and machine learning approaches to advanced NLP and embedding-based models. The proposed methodology aims to overcome the limitations of earlier systems by utilizing MiniLM embeddings for contextual text representation, Deep Neural Networks (DNN) for effective feature extraction, and K- Nearest Neighbors (KNN) for accurate classification. By integrating these techniques with multi-intent annotated datasets, the system provides a more reliable, scalable, and efficient solution for automated customer support through conversational AI.

2.2 DEFINITION OF PROBLEM STATEMENT

`The primary goal of this project is to develop an efficient conversational AI system capable of accurately understanding and classifying customer queries from a customer support dataset with multi-intent annotations. The system focuses on identifying multiple intents within a single query using Natural Language Processing (NLP) techniques. By utilizing MiniLM embeddings for contextual representation, Deep Neural Networks (DNN) for feature selection, and K-Nearest Neighbors (KNN) for classification, the system aims to improve intent detection accuracy and enhance automated customer support services.

2.3 EXISTING SYSTEM

Existing conversational AI systems for customer support primarily rely on traditional Natural Language Processing techniques and machine learning classifiers for intent detection. Many systems use basic text preprocessing methods such as tokenization, stop-word removal, and feature extraction techniques like Bag-of-Words and TF-IDF to convert textual queries into numerical representations. These features are then processed using machine learning algorithms such as Naïve Bayes, Support Vector Machines (SVM), Decision Trees, and K-Nearest Neighbours (KNN) to classify user queries into predefined categories.

Some advanced systems also utilize word embedding techniques such as Word2Vec and GloVe to improve semantic understanding of customer queries. Additionally, rule-based chatbot systems are commonly used in customer support applications, where predefined patterns and keywords are matched with user queries to generate responses. However, these approaches often struggle with complex queries that contain multiple intents within a single message.

Recent developments have introduced transformer-based language models that generate contextual embeddings for text data, improving the understanding of user queries. However, many existing systems do not fully utilize these advanced embedding techniques or hybrid classification approaches, limiting their overall effectiveness in real-world conversational AI applications.

Limitations of Existing System

Despite advancements in conversational AI systems, existing approaches face several limitations when applied to multi-intent customer support datasets:

- Limited contextual understanding: Traditional machine learning models based on Bag-of-Words or TF-IDF representations fail to capture the contextual meaning of words within a sentence, leading to incorrect intent classification.

- Difficulty in handling multi-intent queries: Most existing systems are designed for single-intent classification, making it difficult to accurately interpret queries that contain multiple requests or intentions.

- Class imbalance in datasets: Customer support datasets often contain uneven distributions of query categories, causing classification models to become biased toward frequently occurring intents while ignoring rare ones.

- Inefficient feature representation: Traditional word embedding methods may not fully capture contextual relationships between words, reducing the effectiveness of intent detection models.

- Scalability and performance issues: As customer support datasets grow larger, traditional machine learning models struggle to maintain high accuracy and efficient response times, especially in real-time chatbot applications.

2.4 PROPOSED SYSTEM

The proposed system introduces an intelligent conversational AI framework designed to analyze and classify customer queries using a customer support dataset with multi-intent annotations. The system utilizes Natural Language Processing (NLP) techniques to preprocess textual queries by performing tasks such as tokenization, stop-word removal, and text normalization. After preprocessing, MiniLM embeddings are used to generate contextual vector representations of customer queries, allowing the system to better understand the semantic meaning of text.

To improve classification performance, the dataset is balanced using Synthetic Minority Over-sampling Technique (SMOTE), which addresses class imbalance by generating additional samples for underrepresented intent categories.

Finally, the processed features are classified using the K-Nearest Neighbors (KNN) algorithm, which predicts the intent of customer queries based on similarity with previously labeled data.This hybrid approach improves classification accuracy and enhances automated customer support systems by enabling faster

Advantages of the Proposed System:

The proposed system significantly improves upon the existing approaches by addressing key limitations:

- Improved Intent Detection Accuracy: MiniLM embeddings capture contextual relationships within text, improving the system's ability to accurately understand customer queries.
- Balanced Dataset Handling: The use of SMOTE helps address class imbalance in the dataset, improving model fairness and classification performance.
- Efficient Feature Selection: The Deep Neural Network (DNN) selects important features from the embedding representations, reducing noise and improving prediction accuracy.

2.5 OBJECTIVES

- Develop an Intelligent Conversational AI System – Build a system capable of understanding and classifying customer queries from a multi-intent customer support dataset.
- Improve Intent Detection Accuracy– Use MiniLM embeddings and advanced NLP

Customer Support Dataset With Multi-Intent Annotations for Conversational AI techniques to better capture semantic relationships in text.

- Handle Multi-Intent Queries– Enable the system to identify multiple intents within a single customer query.
- Enhance Classification Performance — Use Deep Neural Networks for feature selection and KNN for accurate intent classification.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS:

Hardware interfaces specify the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements:

- Processor : Intel Core i
- Memory : 1TB.
- RAM : 8GB.

2.6.2 SOFTWARE REQUIREMENTS:

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

- Operating system : Windows 10
- Language : Python
- Backend : Flask Web Application

3. SYSTEM ARCHITECTURE & DESIGN

3. SYSTEM ARCHITECTURE & DESIGN

Project architecture begins with textual preprocessing before generating contextual embeddings using MiniLM. These embeddings are then processed through a Deep Neural Network for feature selection, which identifies the most discriminative dimensions for intent recognition. The refined features are passed into a KNN classifier that efficiently assigns both intent-level outputs and category-level outputs This hierarchical classification ensures that even if a fine-grained intent prediction is uncertain, the broader category can still be correctly identified, improving reliability in chatbot responses.

3.1 PROJECT ARCHITECTURE

This project architecture shows **Architecture** integrates a MiniLM-based word embedding feature extractor with a DNN-driven feature selection module followed by a KNN classifier to accurately predict both intent and category from multi-intent customer support queries. This trained pipeline is deployed within a Flask-based web application that serves as the backend engine for real-time query processing.

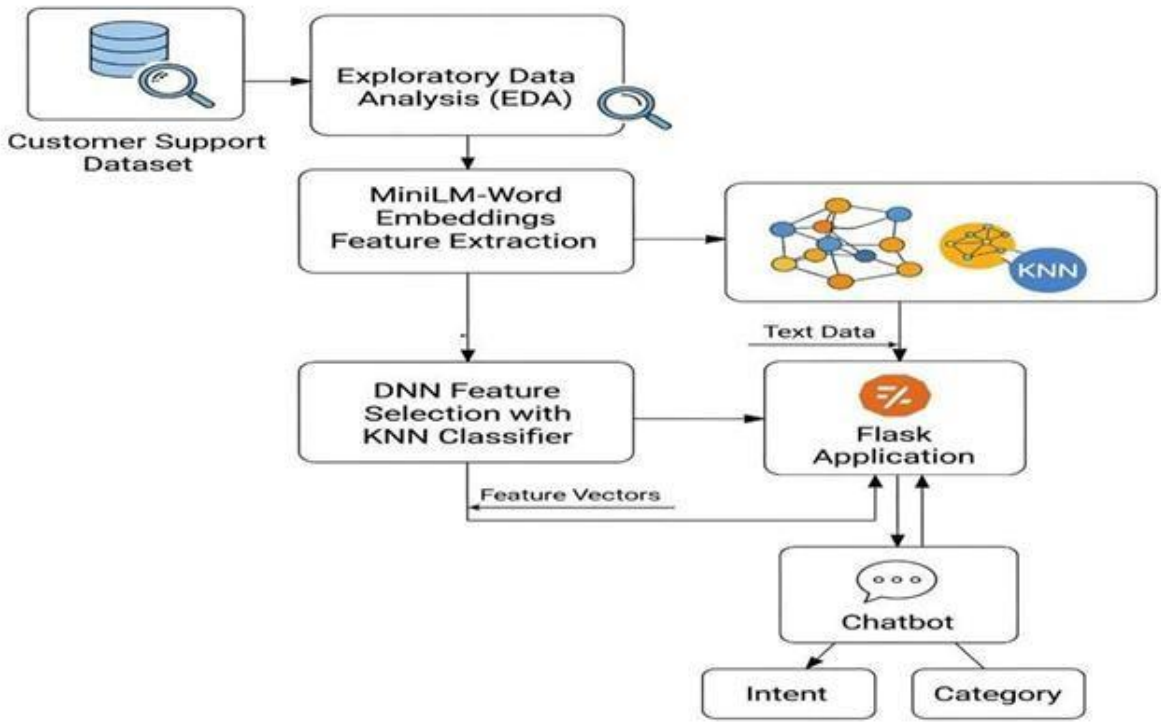

Figure 3.1: Project Architecture of basic intent and Category Applications

3.2 DESCRIPTION

Feature Extraction Module: Converts input text into numerical feature vectors using MiniLM embeddings. Captures semantic and contextual meaning of customer queries.Provides a rich feature base for accurate intent classification.

Feature Selection and Classifiaction: Deep Neural Network (DNN) selects the most relevant features from embeddings. K-Nearest Neighbor (KNN) algorithm classifies the intent based on selected features. Enhances the system's prediction accuracy and efficiency.

Flask Application : Acts as the backend framework connecting all components. Manages communication between the model and the chatbot interface. Ensures smooth data flow and real-time response generation.

Chatbot Application: Serves as the user interaction interface for query input and response display.Utilizes classified intents and categories to respond intelligently. Provides automated and AI-driven customer support.

Exploratory Data Analysis (EDA) : Analyzes and visualizes the customer support dataset before training. Helps identify patterns, correlations, and data distribution. Ensures high-quality and balanced data for feature extraction.

Training and Evaluation : training is where the model *learns* to recognize multiple intents using annotated data. e**valuation** is where the model is *tested* on unseen data to check how accurately it predicts those intents.

Feedback : User feedback on the Customer Support dataset is integrated into the system as ground truth data. This feedback is crucial for Mini LM embeddings algorithms and improving the accuracy of future analysis.

3.3 UML DIAGRAMS

Unified Modeling Language (UML) is a standardized visual language used to model, design, and document the architecture of software systems. It provides a set of graphical notations to represent the structure and behavior of a system, making complex systems easier to understand and communicate among developers, stakeholders, and business analysts.

3.3.1 CLASS DIAGRAM

The class diagram models how the system components collaborate to deliver accurate and fast customer support that improves user satisfaction. The FlaskApp class acts as the controller, handling requests from users and routing them to services like IntentClassifier, AIService, and CustomerServiceClassifier. The IntentClassifier detects multiple intents from each message, while the AIService generates context-aware replies. Conversation data is stored inside ConversationManager and AnalyticsManager, which continuously monitor response time, active sessions, and intent distribution to evaluate service quality

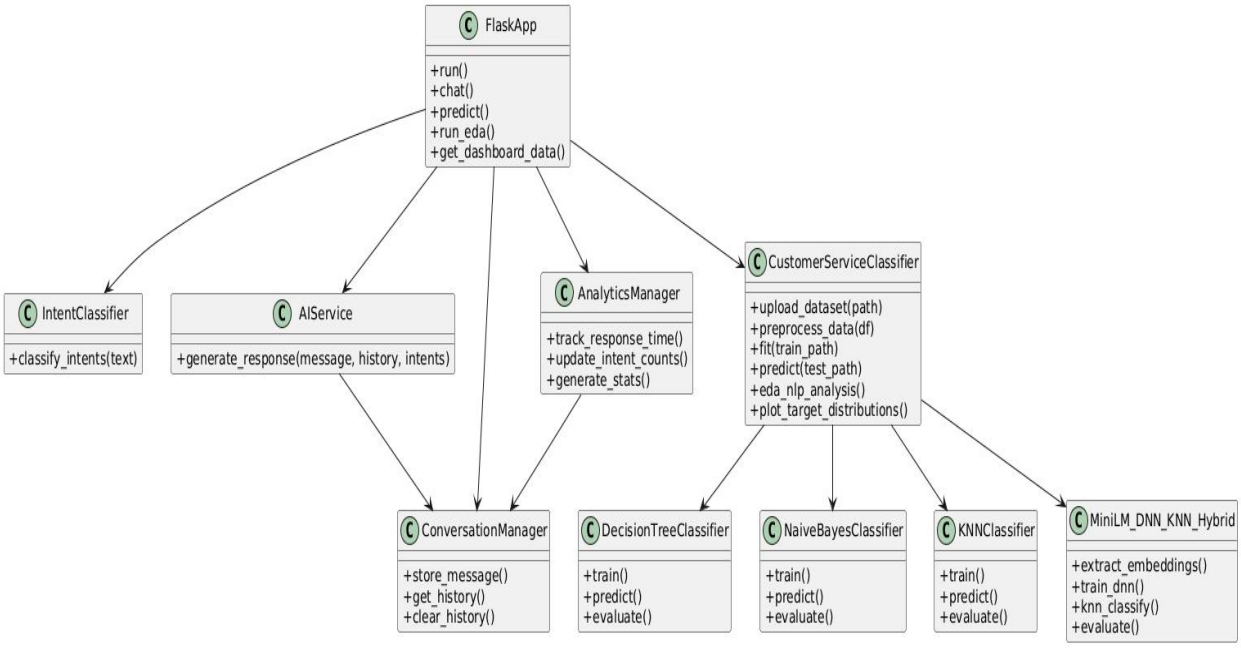

Fig.3.3.1: Class Diagram of Customer Support Dataset With Multi-Intent Annotations for Conversational AI

3.3.2 USE CASE DIAGRAM

The use case diagram represents how different users interact with the system to achieve efficient and intelligent customer support. The primary actor is the Customer, who sends queries, receives automated responses, and views conversation results. The system internally performs intent detection, multi-model classification, and AI-based response generation to resolve issues quickly. Supporting this, the Admin/Analyst actor monitors predictions, evaluates model performance, views analytics dashboards, and runs EDA to understand dataset behavior, ensuring continuous improvement of service quality and overall user satisfaction.

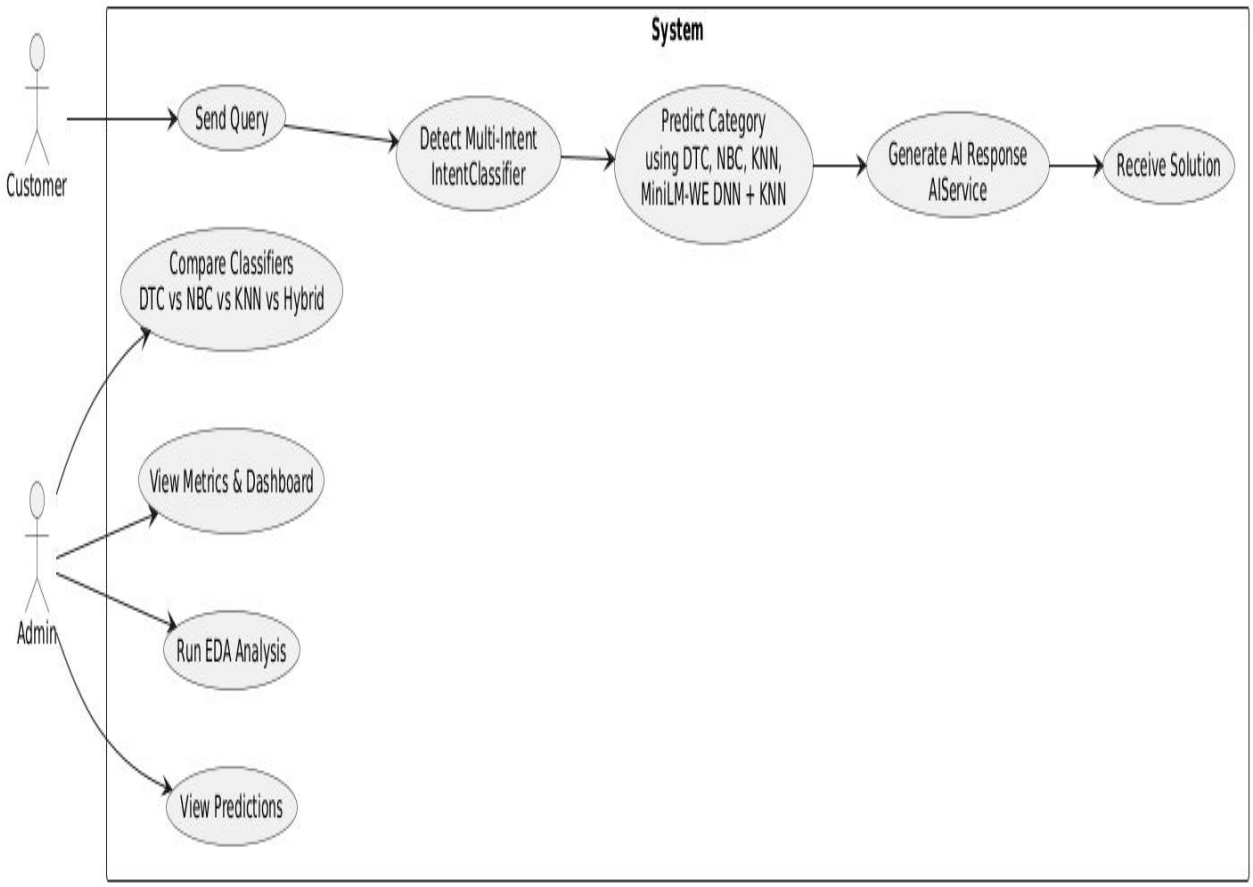

Fig. 3.3.2: Use Case Diagram of Customer Support Dataset With Multi-Intent Annotations for Conversational AI

3.3.3 SEQUENCE DIAGRAM

The sequence diagram represents a simple interaction flow that focuses only on the essential components involved in satisfying the customer's request. When the User sends a support query, it is first processed by the Intent Classifier, which detects one or more intents from the text. The request is then forwarded to the Customer Service Classifier, where preprocessing and feature extraction are performed before classification. This early identification of the problem type helps the system quickly understand whether the issue is technical, billing, account, product, or general, enabling faster and more relevant assistance to improve user satisfaction.

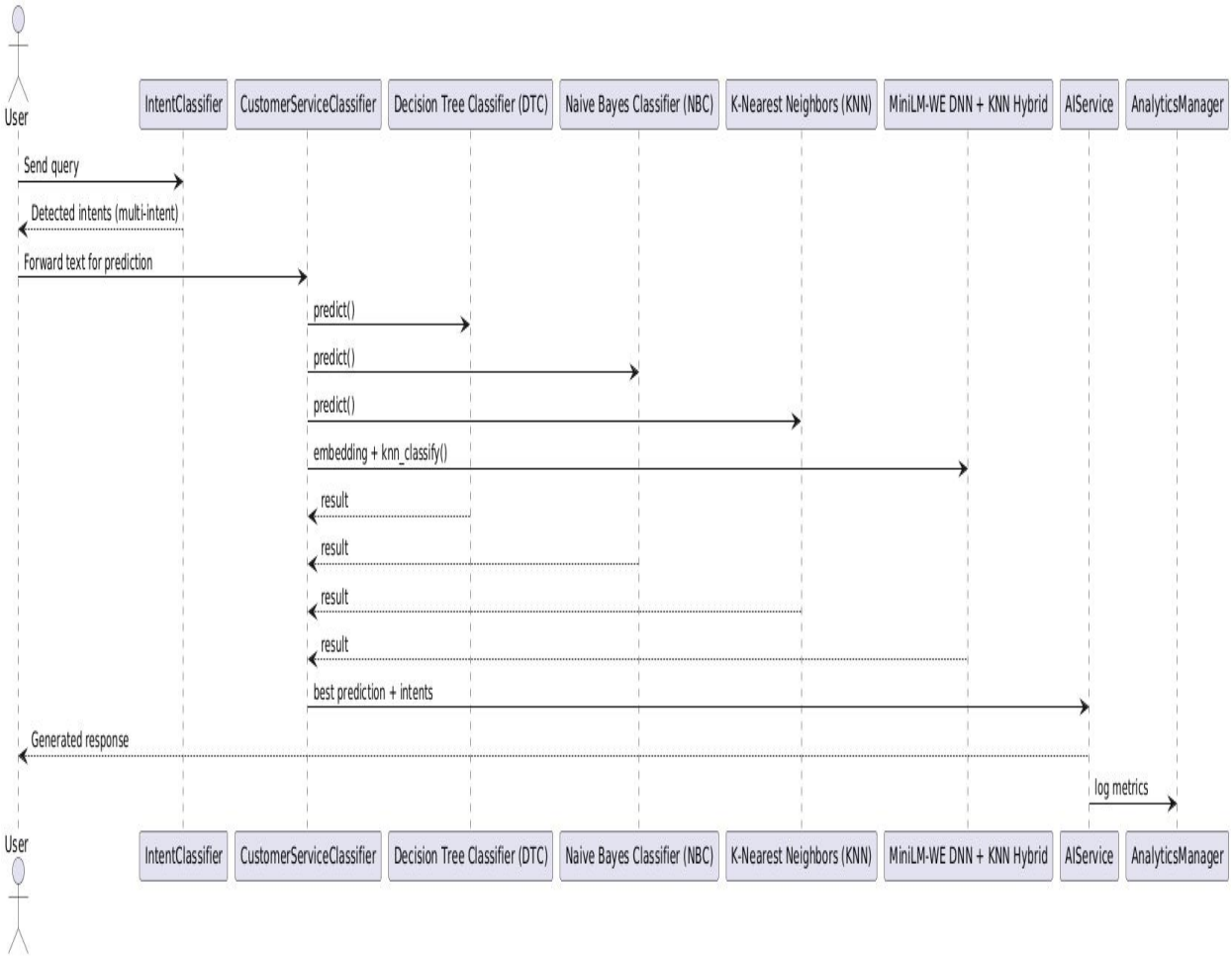

Fig. 3.3.3: Sequence Diagram of Customer Support Dataset With Multi-Intent Annotations for Conversational AI

3.3.4 ACTIVITY DIAGRAM

The activity diagram illustrates the step-by-step workflow followed to process a customer query and deliver a satisfactory solution. The process begins when the User submits a message, after which the system performs text preprocessing such as cleaning and tokenization. The IntentClassifier then detects single or multiple intents to understand the nature of the issue. Based on this understanding, the request moves to the CustomerServiceClassifier, where features are extracted and prepared for prediction, ensuring that the system correctly interprets the customer's problem before generating a response.

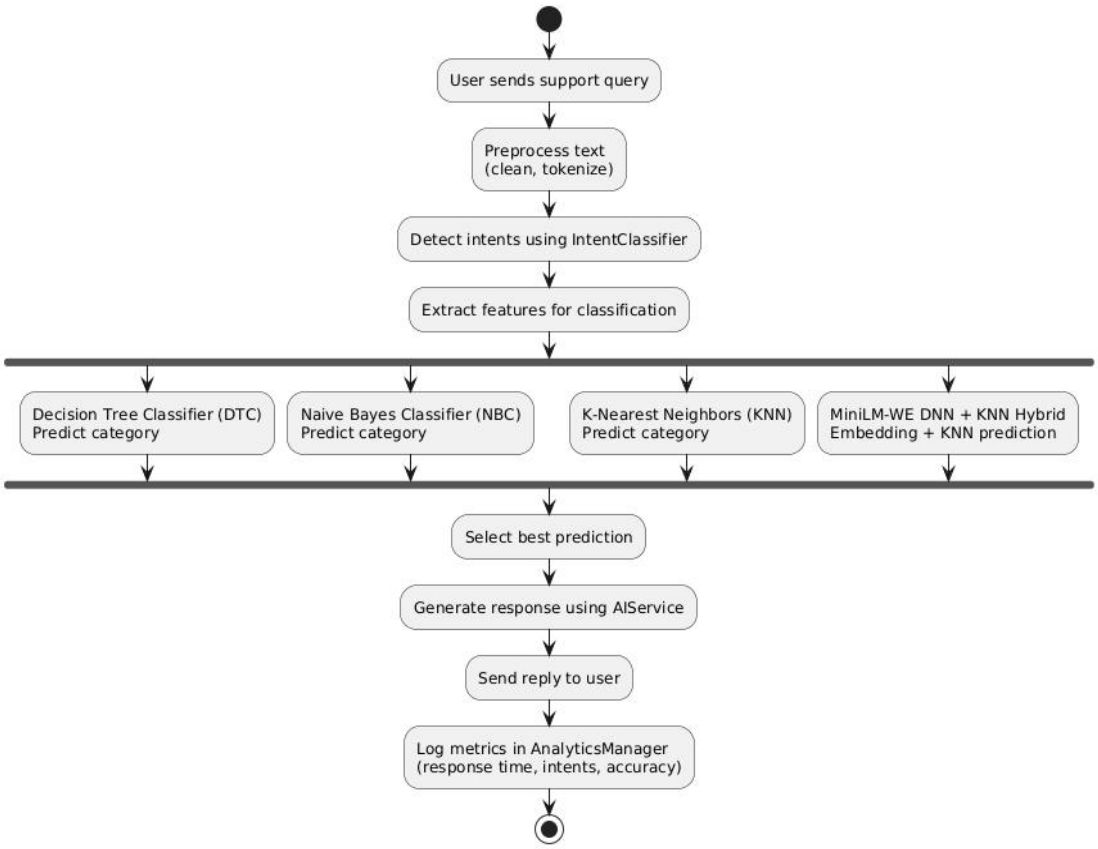

Fig. 3.3.4: Sequence Diagram of Customer Support Dataset With Multi-Intent Annotations for Conversational AI

4. IMPLEMENTATION

4 IMPLEMENTATION

The implementation phase of a project involves executing the planned strategies and tasks. It requires meticulous coordination, resource allocation, and monitoring to ensure that objectives are met efficiently. Effective implementation is crucial for achieving project goals and delivering expected outcomes within the set timeline and budget constraints.

4.1 ALGORITHMS USED

Transformer-Based Models for Intent Detection

Transformer-based models are widely used in conversational AI for understanding user queries and detecting intents. Models such as BERT, RoBERTa, and DistilBERT analyze textual data to identify user intentions in customer support conversations. These models are highly effective at capturing contextual relationships between words and phrases in a sentence.

Transformer models process input text using attention mechanisms, allowing them to understand complex queries that may contain multiple intents. In this project, BERT is used to extract contextual features from customer queries. These features help the system recognize multiple intents within a single query and improve the overall performance of the conversational AI system

Advantages of Transformer-Based Models:

- Highly effective at understanding contextual relationships in text.
- Can detect multiple intents within a single customer query.
- Pre-trained models reduce the need for very large labeled datasets.

Disadvantages of Transformer-Based Models:

- Require high computational resources for training and inference.
- Large model size increases memory consumption.

RNN-Based Models for Sequential Text Understanding

To further enhance intent detection, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are used to process sequential text data. In this project, BiLSTM (Bidirectional Long Short-Term Memory) is used to capture contextual dependencies in customer queries.

BiLSTM processes text sequences in both forward and backward directions, enabling the system to understand the relationship between words more effectively. This helps the chatbot interpret complex customer queries such as requests that include multiple actions like checking order status and requesting refunds.

Advantages of RNN-Based Models:

- Effective in capturing sequential dependencies in text data.
- Improves contextual understanding of user queries.
- BiLSTM processes information in both directions, enhancing context aware

Disadvantages of RNN-Based Models:

- Computationally expensive for large datasets.
- Training time is longer compared to traditional machine learning models.
- May suffer from vanishing gradient problems in long sequences.

1. BERT for Contextual Feature Extraction

- BERT is a transformer-based deep learning model widely used for natural language understanding tasks.

2. BiLSTM for Sequential Context Modeling

- BiLSTMs analyze sentences in both forward and backward directions.
- Helps the system understand the relationship between multiple intents present in the same sentence.

3. Attention Mechanism for Important Keyword Identification

- The attention mechanism assigns different weights to frames, ensuring the model focuses on critical content rather than treating all frames equally.

4.2 SAMPLE CODE

```
import os
import glob
import uuid
import logging
import matplotlib
matplotlib.use('Agg')  # Use non-interactive backend for matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from datetime import datetime

from flask import Flask, render_template, request, jsonify, session, redirect, url_for
from flask_cors import CORS

# Import classifiers and services
from customer_service_classifier import CustomerServiceClassifier
from ai_service import AIService
from intent_classifier import IntentClassifier

# ==============================
# Configure logging
# ==============================
logging.basicConfig(level=logging.DEBUG)

app = Flask(__name__)
app.secret_key = os.environ.get("SESSION_SECRET", "dev-secret-key")
CORS(app)

# ==============================
# Initialize AI services
# ==============================
ai_service = AIService()
intent_classifier = IntentClassifier()

# ==============================
# Initialize CustomerServiceClassifier
# ==============================
classifier = CustomerServiceClassifier(model_dir="model")
train_path = "Dataset/Bitext_Sample_Customer_Service_Training_Dataset.csv"
test_path = "Dataset/Bitext_Sample_Customer_Service_Testing_Dataset.csv"

# ==============================
# In-memory storage
```

```
# ==============================
conversations = {}
analytics_data = {
    'total_conversations': 0,
    'response_times': [],
    'intent_counts': {'technical': 0, 'billing': 0, 'account': 0, 'product': 0, 'general': 0},
    'conversation_timeline': [],
    'active_sessions': set()
}

# ==============================
# Template Filter for Footer
# ==============================
@app.template_filter('datetimeformat')
def datetimeformat(value, format='%Y'):
    if value == 'now':
        return datetime.now().strftime(format)
    return value

# ==============================
# Routes
# ==============================
@app.route('/')
def index():
    return render_template('home.html')

@app.route('/dashboard')
def dashboard():
    return render_template('dashboard.html')

@app.route('/visuallongation')
def visualization():
    return render_template('visualization.html')

@app.route('/eda')
def eda_page():
    return render_template('eda.html')

@app.route('/prediction')
def predict_page():
    return render_template('predictions.html')

# ==============================
# EDA Endpoint
# ==============================
```

```
@app.route('/api/eda', methods=['GET'])
def run_eda():
    try:
        df = classifier.upload_dataset(train_path)
        X, Y_dict = classifier.preprocess_data(df, target_cols=["intent", "category"])

        eda_plot_dir = os.path.join(app.static_folder, 'eda_plots')
        os.makedirs(eda_plot_dir, exist_ok=True)

        original_show = plt.show
        def save_plot():
            plot_filename = f"plot_{uuid.uuid4()}.png"
            plot_path = os.path.join(eda_plot_dir, plot_filename)
            plt.savefig(plot_path, bbox_inches='tight')
            plt.close()
            return os.path.join('eda_plots', plot_filename)
        plt.show = save_plot

        plot_paths = []
        classifier.eda_nlp_analysis(X, num_words=100, top_n_words=20)
        plot_paths.extend([
            os.path.join('eda_plots', f) for f in os.listdir(eda_plot_dir)
            if f.startswith('plot_') and f.endswith('.png')
        ])

        classifier.plot_target_distributions(Y_dict)
        plot_paths.extend([
            os.path.join('eda_plots', f) for f in os.listdir(eda_plot_dir)
            if f.startswith('plot_') and f.endswith('.png') and f not in plot_paths
        ])

        plt.show = original_show
        return jsonify({'plots': plot_paths})

    except Exception as e:
        logging.error(f"Error in EDA endpoint: {str(e)}")
        return jsonify({'error': 'Failed to generate EDA visualizations', 'details': str(e)}), 500

# ==============================
# Prediction Endpoint
# ==============================
@app.route('/api/predict', methods=['GET'])
def predict():
    try:
        # Ensure the classifier is trained
```

```
        if not classifier.models:
            classifier.fit(train_path, target_cols=["intent", "category"])

        # Run predictions using the test dataset
        predictions_df = classifier.predict(test_path, model_base_name="MiniLM-WE DNN")

        # Convert predictions to a list of dictionaries for JSON response
        predictions = predictions_df.to_dict(orient='records')

        return jsonify({
            'success': True,
            'predictions': predictions,
            'columns': list(predictions_df.columns)
        })

    except Exception as e:
        logging.error(f"Error in prediction endpoint: {str(e)}")
        return jsonify({'error': 'Failed to generate predictions', 'details': str(e)}), 500

# ==============================
# Metrics Endpoints (DTC, NBC, KNN, DNN-KNN)
# ==============================
@app.route('/api/dtc', methods=['GET'])
def dtc_metrics():
    try:
        results_dir = os.path.join(app.static_folder, 'results', 'dtc')
        targets = ['intent', 'category']
        response = {}
        for target in targets:
            target_dir = os.path.join(results_dir, target)
            png_files = glob.glob(os.path.join(target_dir, '*.png')) if os.path.exists(target_dir)
else []
            relative_paths = [os.path.join('results', 'dtc', target,
os.path.basename(f)).replace(os.sep, '/')
                    for f in png_files]
            confusion_matrices = [p for p in relative_paths if 'confusion_matrix' in p.lower()]
            roc_curves = [p for p in relative_paths if 'roc_curve' in p.lower()]
            other_files = [p for p in relative_paths if p not in confusion_matrices and p not in
roc_curves]
            response[target] = {
                'confusion_matrices': confusion_matrices,
                'roc_curves': roc_curves,
                'other_plots': other_files
            }
        return jsonify(response)
```

```
    except Exception as e:
        logging.error(f"Error in DTC metrics endpoint: {str(e)}")
        return jsonify({'error': 'Failed to retrieve DTC metrics', 'details': str(e)}), 500

@app.route('/api/nbc', methods=['GET'])
def nbc_metrics():
    try:
        results_dir = os.path.join(app.static_folder, 'results', 'nbc')
        targets = ['intent', 'category']
        response = {}
        for target in targets:
            target_dir = os.path.join(results_dir, target)
            png_files = glob.glob(os.path.join(target_dir, '*.png')) if os.path.exists(target_dir)
else []
            relative_paths = [os.path.join('results', 'nbc', target,
os.path.basename(f)).replace(os.sep, '/')
                    for f in png_files]
            confusion_matrices = [p for p in relative_paths if 'confusion_matrix' in p.lower()]
            roc_curves = [p for p in relative_paths if 'roc_curve' in p.lower()]
            other_files = [p for p in relative_paths if p not in confusion_matrices and p not in
roc_curves]
            response[target] = {
                'confusion_matrices': confusion_matrices,
                'roc_curves': roc_curves,
                'other_plots': other_files
            }
        return jsonify(response)
    except Exception as e:
        logging.error(f"Error in NBC metrics endpoint: {str(e)}")
        return jsonify({'error': 'Failed to retrieve NBC metrics', 'details': str(e)}), 500

@app.route('/api/knn', methods=['GET'])
def knn_metrics():
    try:
        results_dir = os.path.join(app.static_folder, 'results', 'knn')
        targets = ['intent', 'category']
        response = {}
        for target in targets:
            target_dir = os.path.join(results_dir, target)
            png_files = glob.glob(os.path.join(target_dir, '*.png')) if os.path.exists(target_dir)
else []
            relative_paths = [os.path.join('results', 'knn', target,
os.path.basename(f)).replace(os.sep, '/')
                    for f in png_files]
            confusion_matrices = [p for p in relative_paths if 'confusion_matrix' in p.lower()]
```

```
            roc_curves = [p for p in relative_paths if 'roc_curve' in p.lower()]
            other_files = [p for p in relative_paths if p not in confusion_matrices and p not in
roc_curves]
            response[target] = {
                'confusion_matrices': confusion_matrices,
                'roc_curves': roc_curves,
                'other_plots': other_files
            }
        return jsonify(response)
    except Exception as e:
        logging.error(f"Error in KNN metrics endpoint: {str(e)}")
        return jsonify({'error': 'Failed to retrieve KNN metrics', 'details': str(e)}), 500

@app.route('/api/dnn_knn', methods=['GET'])
def dnn_knn_metrics():
    try:
        results_dir = os.path.join(app.static_folder, 'results', 'MiniLM-WE DNN KNN')
        targets = ['intent', 'category']
        response = {}
        for target in targets:
            target_dir = os.path.join(results_dir, target)
            png_files = glob.glob(os.path.join(target_dir, '*.png')) if os.path.exists(target_dir)
else []
            relative_paths = [os.path.join('results', 'MiniLM-WE DNN KNN', target,
os.path.basename(f)).replace(os.sep, '/')
                              for f in png_files]
            confusion_matrices = [p for p in relative_paths if 'confusion_matrix' in p.lower()]
            roc_curves = [p for p in relative_paths if 'roc_curve' in p.lower()]
            other_files = [p for p in relative_paths if p not in confusion_matrices and p not in
roc_curves]
            response[target] = {
                'confusion_matrices': confusion_matrices,
                'roc_curves': roc_curves,
                'other_plots': other_files
            }
        return jsonify(response)
    except Exception as e:
        logging.error(f"Error in DNN-KNN metrics endpoint: {str(e)}")
        return jsonify({'error': 'Failed to retrieve DNN-KNN metrics', 'details': str(e)}), 500

# ==============================
# Classifiers page + redirects
# ==============================
@app.route('/classifiers')
def classifiers_page():
```

```
    classifier_name = request.args.get('classifier', 'dtc')
    if classifier_name not in ['dtc', 'nbc', 'knn', 'dnn_knn']:
        classifier_name = 'dtc'
    return render_template('classifiers.html', classifier=classifier_name)

@app.route('/dtc')
def dtc_redirect():
    return redirect(url_for('classifiers_page', classifier='dtc'))

@app.route('/nbc')
def nbc_redirect():
    return redirect(url_for('classifiers_page', classifier='nbc'))

@app.route('/knn')
def knn_redirect():
    return redirect(url_for('classifiers_page', classifier='knn'))

@app.route('/dnn_knn')
def dnn_knn_redirect():
    return redirect(url_for('classifiers_page', classifier='dnn_knn'))

# ==============================
# Chat Endpoints
# ==============================
@app.route('/chat')
def chat_page():
    if 'session_id' not in session:
        session['session_id'] = str(uuid.uuid4())
        conversations[session['session_id']] = []
        analytics_data['active_sessions'].add(session['session_id'])
    return render_template('chat.html')

@app.route('/api/chat', methods=['POST'])
def chat():
    try:
        start_time = datetime.now()
        data = request.get_json()
        user_message = data.get('message', '').strip()
        if not user_message:
            return jsonify({'error': 'Message cannot be empty'}), 400

        session_id = session.get('session_id')
        if not session_id:
            return jsonify({'error': 'Session not found'}), 400
        if session_id not in conversations:
```

```
            conversations[session_id] = []
            analytics_data['active_sessions'].add(session_id)

        user_msg = {'role': 'user', 'content': user_message, 'timestamp': datetime.now().isoformat()}
        conversations[session_id].append(user_msg)

        detected_intents = intent_classifier.classify_intents(user_message)
        ai_response = ai_service.generate_response(user_message, conversations[session_id], detected_intents)

        response_time = (datetime.now() - start_time).total_seconds() * 1000
        analytics_data['total_conversations'] += 1
        analytics_data['response_times'].append(response_time)
        if len(analytics_data['response_times']) > 100:
            analytics_data['response_times'] = analytics_data['response_times'][-100:]

        if detected_intents:
            primary_intent = detected_intents[0]['intent']
            analytics_data['intent_counts'][primary_intent] = analytics_data['intent_counts'].get(primary_intent, 0) + 1

        analytics_data['conversation_timeline'].append({
            'timestamp': datetime.now().isoformat(),
            'response_time': response_time
        })
        if len(analytics_data['conversation_timeline']) > 1440:
            analytics_data['conversation_timeline'] = analytics_data['conversation_timeline'][-1440:]

        ai_msg = {
            'role': 'assistant',
            'content': ai_response,
            'timestamp': datetime.now().isoformat(),
            'intents': detected_intents
        }
        conversations[session_id].append(ai_msg)

        return jsonify({'response': ai_response, 'intents': detected_intents, 'timestamp': ai_msg['timestamp']})
    except Exception as e:
        logging.error(f"Error in chat endpoint: {str(e)}")
        return jsonify({'error': 'Error processing message', 'details': str(e) if app.debug else None}), 500
```

```
@app.route('/api/conversation', methods=['GET'])
def get_conversation():
    try:
        session_id = session.get('session_id')
        if not session_id or session_id not in conversations:
            return jsonify({'messages': []})
        return jsonify({'messages': conversations[session_id]})
    except Exception as e:
        logging.error(f"Error getting conversation: {str(e)}")
        return jsonify({'error': 'Failed to retrieve conversation'}), 500

@app.route('/api/clear', methods=['POST'])
def clear_conversation():
    try:
        session_id = session.get('session_id')
        if session_id and session_id in conversations:
            conversations[session_id] = []
        return jsonify({'success': True})
    except Exception as e:
        logging.error(f"Error clearing conversation: {str(e)}")
        return jsonify({'error': 'Failed to clear conversation'}), 500

# ==============================
# Stats & Analytics Endpoints
# ==============================
@app.route('/api/stats')
def get_stats():
    """Get basic system statistics"""
    return jsonify({
        'total_chats': analytics_data['total_conversations'],
        'avg_response_time': int(sum(analytics_data['response_times']) / len(analytics_data['response_times'])) if analytics_data['response_times'] else 0,
        'active_users': len(analytics_data['active_sessions']),
        'system_uptime': 99.9
    })

@app.route('/api/dashboard-data')
def get_dashboard_data():
    """Get dashboard analytics data"""
    import random
    from datetime import datetime, timedelta

    # Generate timeline data for last 24 hours
    now = datetime.now()
    timeline_labels = []
```

```
    timeline_data = []

    for i in range(24):
        hour = now - timedelta(hours=i)
        timeline_labels.insert(0, hour.strftime('%H:00'))
        # Use real data when available, otherwise generate realistic patterns
        base_conversations = len([c for c in analytics_data['conversation_timeline']
                                  if datetime.fromisoformat(c['timestamp']).hour == hour.hour])
        timeline_data.insert(0, base_conversations + random.randint(0, 5))

    return jsonify({
        'active_conversations': len(analytics_data['active_sessions']),
        'avg_response_time': int(sum(analytics_data['response_times']) / len(analytics_data['response_times'])) if analytics_data['response_times'] else random.randint(150, 300),
        'resolved_issues': analytics_data['total_conversations'],
        'satisfaction_score': random.randint(85, 98),
        'conversation_timeline': {
            'labels': timeline_labels,
            'data': timeline_data
        },
        'intent_distribution': [
            analytics_data['intent_counts']['technical'],
            analytics_data['intent_counts']['billing'],
            analytics_data['intent_counts']['account'],
            analytics_data['intent_counts']['product'],
            analytics_data['intent_counts']['general']
        ],
        'recent_conversations': [
            {
                'time': (datetime.now() - timedelta(minutes=i*5)).strftime('%H:%M'),
                'user_id': list(analytics_data['active_sessions'])[i % len(analytics_data['active_sessions'])] if analytics_data['active_sessions'] else f"user_{i}",
                'intent': list(analytics_data['intent_counts'].keys())[i % 5],
                'status': 'resolved' if i % 3 == 0 else 'in_progress',
                'duration': f"{random.randint(2, 15)}m"
            }
            for i in range(min(10, len(analytics_data['active_sessions']) or 5))
        ]
    })

@app.route('/api/analytics-data')
def get_analytics_data():
    """Get detailed analytics data"""
    import random
```

```
from datetime import datetime, timedelta

time_range = request.args.get('range', '24h')

# Generate response time trend data
now = datetime.now()
trend_labels = []
avg_times = []
max_times = []

periods = 24 if time_range == '24h' else (7 if time_range == '7d' else 30)
unit = 'hours' if time_range == '24h' else 'days'

for i in range(periods):
    if unit == 'hours':
        time_point = now - timedelta(hours=i)
        trend_labels.insert(0, time_point.strftime('%H:00'))
    else:
        time_point = now - timedelta(days=i)
        trend_labels.insert(0, time_point.strftime('%m/%d'))

    # Use real response times when available
    period_times = [rt for rt in analytics_data['response_times'][-periods:]]
    if period_times:
        avg_times.insert(0, int(sum(period_times) / len(period_times)))
        max_times.insert(0, int(max(period_times)))
    else:
        avg_times.insert(0, random.randint(200, 400))
        max_times.insert(0, random.randint(500, 800))

return jsonify({
    'current_response_time': analytics_data['response_times'][-1] if analytics_data['response_times'] else random.randint(150, 300),
    'throughput_per_min': random.randint(5, 20),
    'active_connections': len(analytics_data['active_sessions']),
    'error_rate': random.randint(0, 3),
    'memory_usage': random.randint(45, 75),
    'cpu_usage': random.randint(20, 60),
    'response_time_trend': {
        'labels': trend_labels,
        'avg_times': avg_times,
        'max_times': max_times
    },
    'status_distribution': [70, 20, 8, 2],  # resolved, in_progress, pending, escalated
    'confidence_distribution': [65, 25, 10],  # high, medium, low
```

```
        'accuracy_by_intent': [92, 88, 95, 90, 85],  # technical, billing, account, product,
general
        'detailed_analytics': [
            {
                'timestamp': (datetime.now() - timedelta(minutes=i*2)).strftime('%H:%M:%S'),
                'session_id':                    list(analytics_data['active_sessions'])[i                    %
len(analytics_data['active_sessions'])] if analytics_data['active_sessions'] else f"session_{i}",
                'intent': list(analytics_data['intent_counts'].keys())[i % 5],
                'confidence': random.uniform(0.6, 0.95),
                'response_time': random.randint(150, 400),
                'satisfaction': random.randint(3, 5) if i % 3 == 0 else None,
                'status': 'resolved' if i % 2 == 0 else 'in_progress'
            }
            for i in range(50)
        ]
    })

# ==============================
# Error handlers
# ==============================
@app.errorhandler(404)
def not_found(error):
    return render_template('home.html'), 404

@app.errorhandler(500)
def internal_error(error):
    logging.error(f"Internal server error: {str(error)}")
    return jsonify({'error': 'Internal server error'}), 500

# ==============================
# Run App
# ==============================
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

5. RESULTS & DISCUSSION

5. RESULTS & DISCUSSION

The following screenshots showcase the results of our project, highlighting key features and functionalities. These visual representations provide a clear overview of how the system performs under various conditions, demonstrating its effectiveness and user interface. The screenshots serve as a visual aid to support the project's technical and operational achievements.

5.1 Implementation description :

In below screen, the home interface of **ConvoSense**, an AI-powered customer support system. It presents the main features of the platform, which is designed to analyze customer conversations and provide intelligent responses.

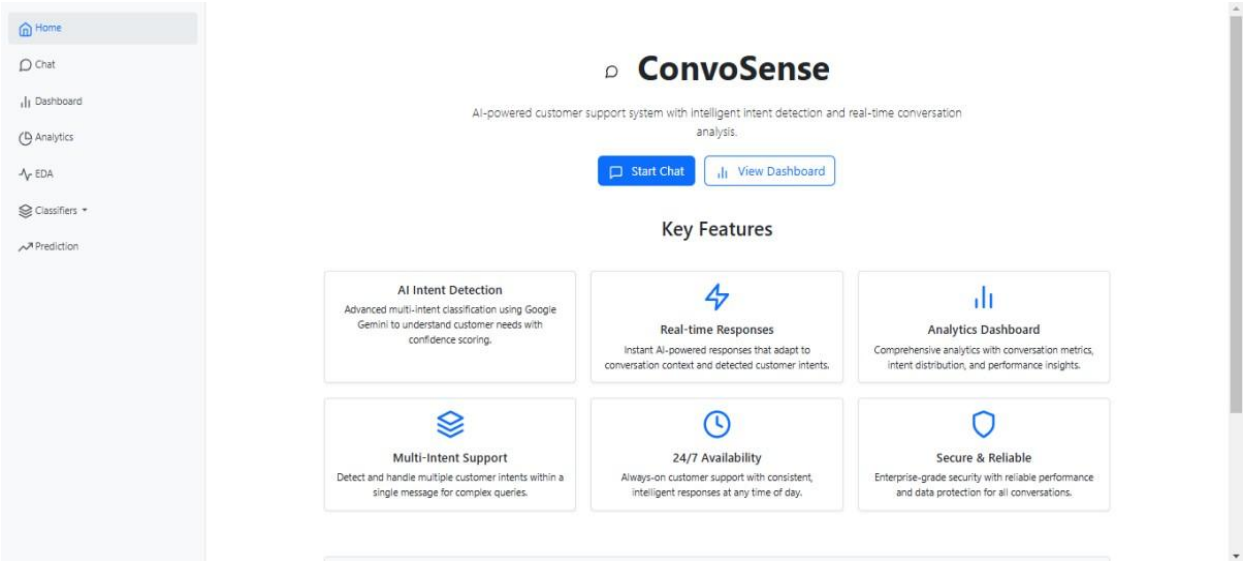

Figure 5 .1 : Home page for Multi-Intent Annotations For Conversational AI.

5.2 Dataset description :

In below screen, the Support Dashboard of the ConvoSense system, which displays analytics and performance details of customer support conversations. The dashboard also includes a conversation timeline graph that shows chat activity over the last 24 hours and an intent distribution chart.

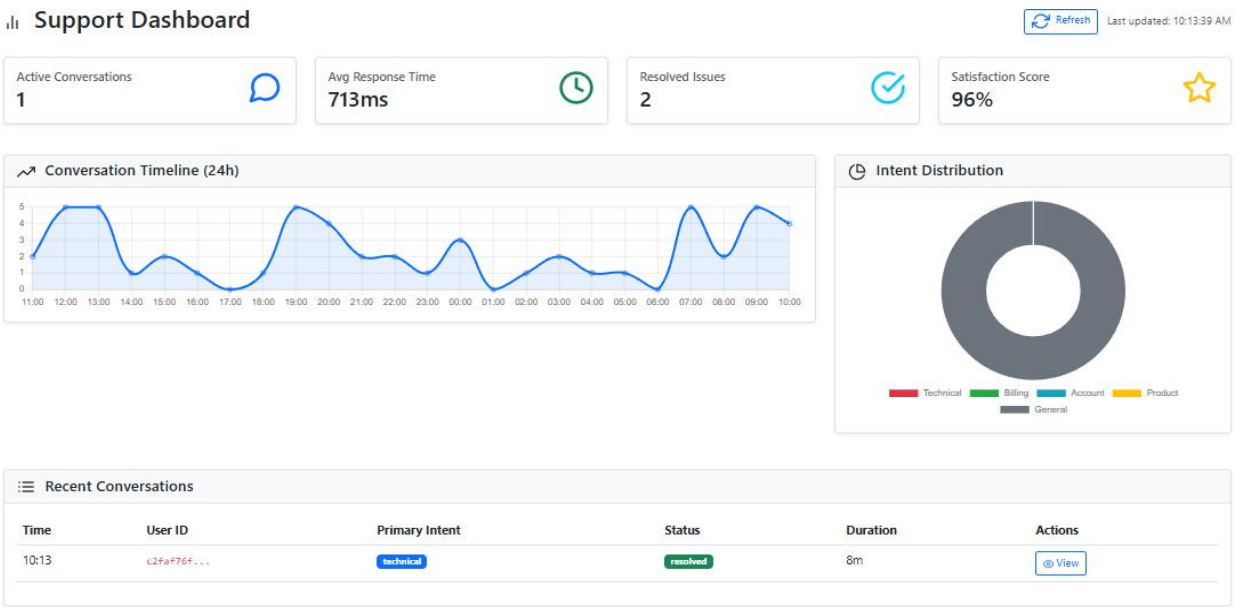

Figure 5.2 : Real-Time AI Support Dashboard for Conversation Monitoring and Analytics.

5.3 Results and Description :

In below screen, depicts the advanced analytics dashboard for in-depth system performance monitoring over the last 24 hours, reporting response time of 651.598ms, throughput of 19 messages per minute, 1 active connection, and a 3% error rate

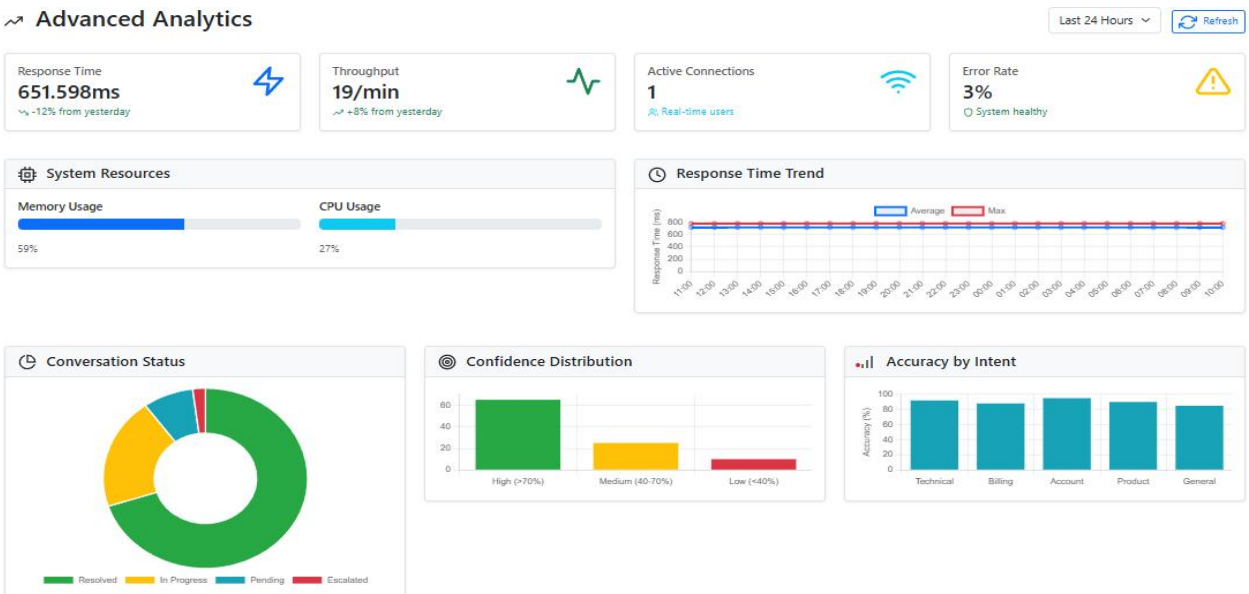

Figure 5.3 : Advanced Analytics Dashboard for AI Support System Performance Monitoring.

5.4 Exploratory Data Analysis :

In below screen, Exploratory Data Analysis (EDA) is the initial step in data analysis where the dataset is examined and explored to understand what information it contains. It involves looking at the data using statistics and visualizations to identify patterns, trends, relationships, errors, or unusual values. The main goal of EDA is to gain a clear understanding of the data before applying machine learning or deep learning models

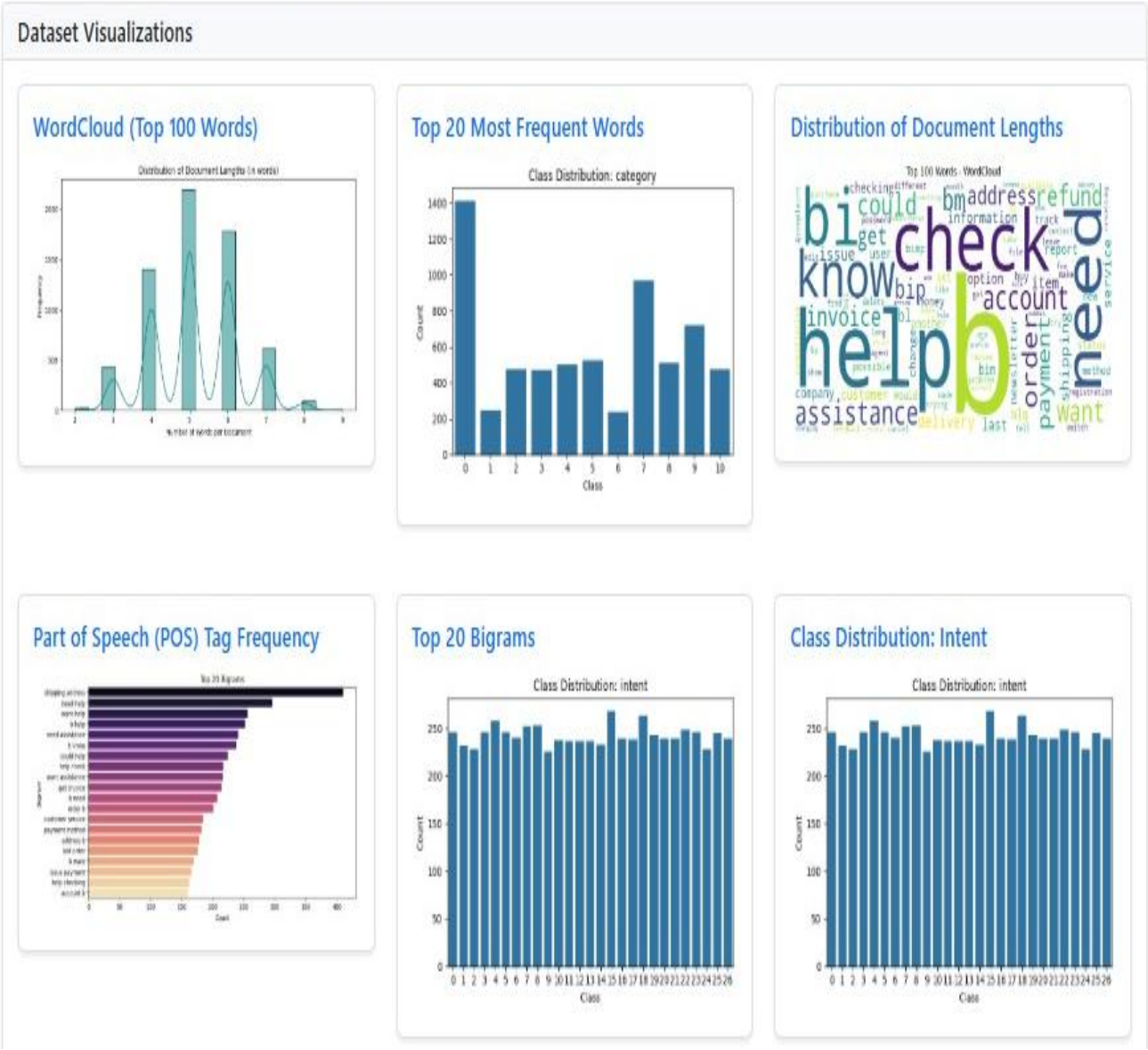

Figure 5.4 : Exploratory Data Analysis of Customer Support Intent Dataset.

5.5 Target Column : Intent Classification :

In below screen, presents confusion matrices for four intent classification models on a multi- class customer support dataset, with true labels (rows) and predicted labels (columns) aligned across identical intent categories such as "check_refund," "track_order," "invoice," "password," and "cancel_order."

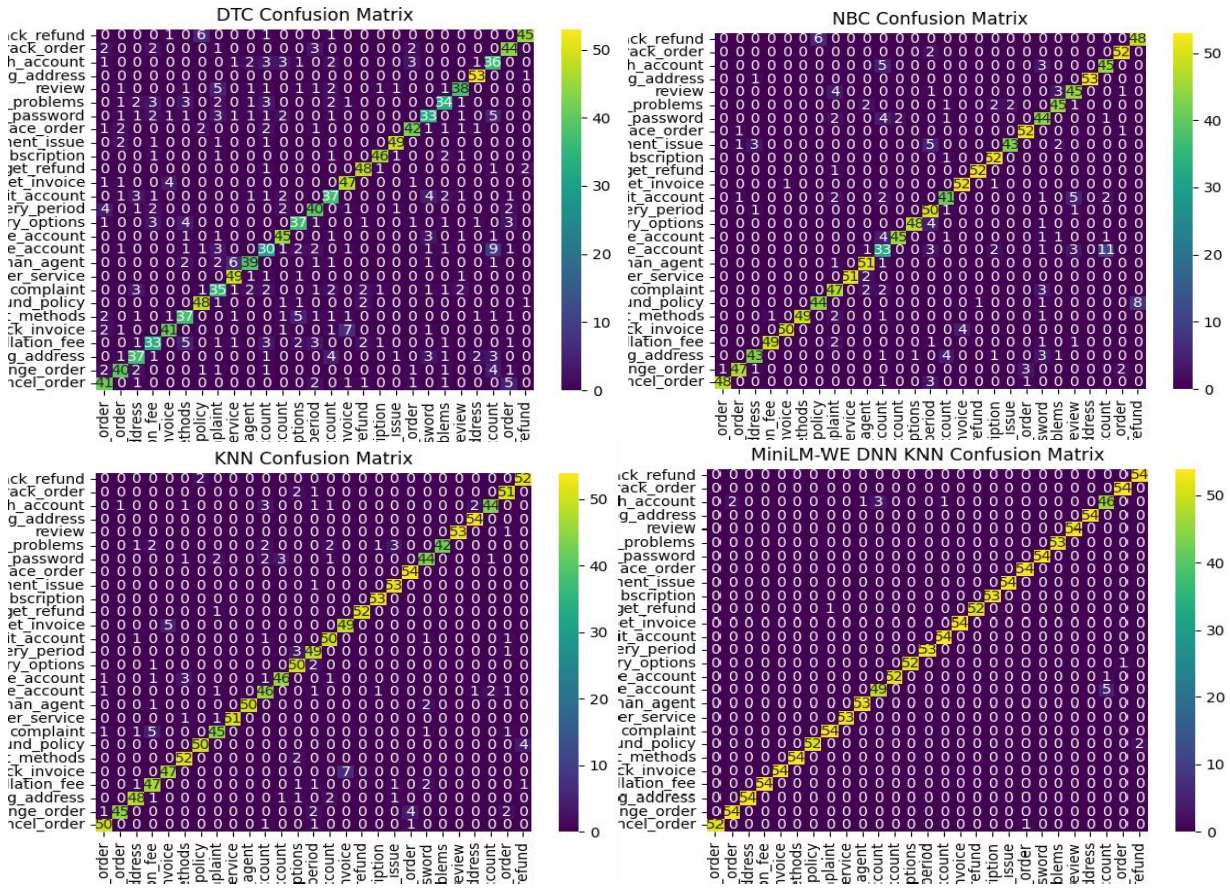

Figure 5.5 : Confusion matrix obtained using (a) DTC model. (b) NBC model. (c) KNN model. (d) Proposed Mini LM-WE KNN model for Intent Classification column.

5.6 Receiver Operating Characteristic (ROC) :

In below screen, presents Receiver Operating Characteristic (ROC) curves for four intent classification models evaluated in a one-vs-rest setting across multiple customer support intents, with True Positive Rate (TPR) plotted against False Positive Rate (FPR).

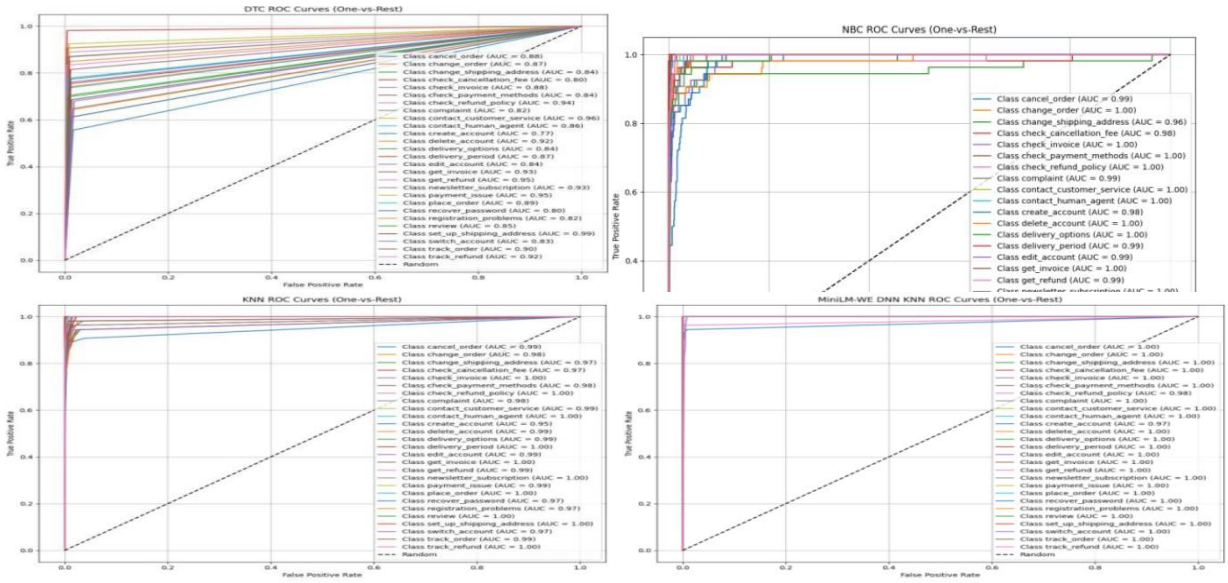

Figure 5.6 : ROC Curve obtained using (a) DTC model. (b) NBC model. (c) KNN model.

5.7 Target Column: Category Classification:

In below graph, presents confusion matrices for four models classifying customer support queries into 10 coarse categories (ADDRESS, REFUND, PAYMENT, ORDER, NEWSLETTER, INVOICE,FEEDBACK, DELIVERY, CONTACT, ACCOUNT), with true labels on rows and predicted labels on columns, and cell values color-coded by count (yellow high, purple low).

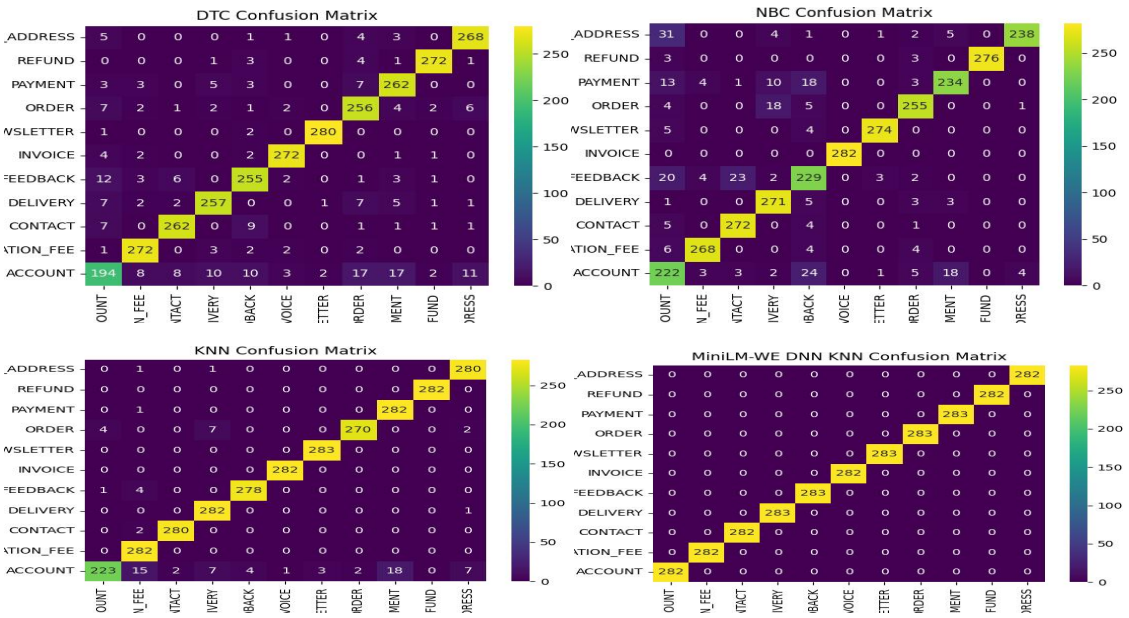

Figure 5.7 : Confusion Matrix obtained using (a) DTC model. (b) NBC model. (c) KNN model. (d) Proposed Mini LM-WE KNN model for Category Classification column.

6. VALIDATION

6.VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the inappropriate content detection system. The testing process involves multiple stages, including dataset validation, model performance evaluation, and real-world testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in detecting inappropriate content while minimizing false positives and false negatives.

6.1 INTRODUCTION

First, First, the customer support dataset is divided into training and testing sets using an 80–20 split. The training data is used to train the machine learning models, while the testing data evaluates how well the system predicts unseen customer queries. To ensure reliability and reduce overfitting, cross-validation techniques are applied so that the model is tested on multiple subsets of the data. This helps confirm that the system can generalize well and perform consistently when handling new queries from users.

The performance of the proposed model is evaluated using important classification metrics such as accuracy, precision, recall, and F1-score. In addition, a confusion matrix is used to analyze correct and incorrect predictions of both intent and category labels. This analysis helps identify where the model performs well and where improvements are needed. The results of the proposed MiniLM-WE + DNN-KNN hybrid model are compared with traditional classifiers such as Decision Tree (DTC), K-Nearest Neighbors (KNN), and Naïve Bayes Classifier (NBC), demonstrating that the proposed method achieves better classification performance.

Finally, the trained model is integrated into a chatbot interface to simulate real-world customer support interactions. The system is tested with new user queries to verify its ability to detect multiple intents and categories accurately in real time.

6.2 TEST CASES

TABLE 6.2.1 UPLOADING DATASET

Test case ID	Test case name	Purpose	Test Case	Output
1	Upload Customer Support Dataset.	To load dataset into the system for processing.	The user uploads the customer support dataset containing queries, intents, and categories.	Dataset successfully loaded and ready for preprocessing.

TABLE 6.2.2 CLASSIFICATION

Test case ID	Test case name	Purpose	Input	Output
1	Intent Prediction	To identify the intent of customer query	Customer query text	Predicted intent label
2	Category Prediction	To classify query into category	Customer query text	Inappropriate Content

7. CONCLUSION & FUTURE ASPECTS

7.CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

7.1 PROJECT CONCLUSION

This The development of the Customer Support Dataset with Multi-Intent Annotations system demonstrates the effectiveness of combining natural language processing, machine learning, and web-based frameworks for intelligent conversational AI. By integrating multiple classifiers — including Decision Tree, Naive Bayes, K- Nearest Neighbors, and the proposed hybrid MiniLM-WE DNN-KNN model — the system can accurately detect and respond to diverse customer intents. The Flask-based implementation ensures a seamless user experience, providing real-time predictions, visualization of model performance, and analytics dashboards that track conversation metrics such as response time, intent distribution, and user satisfaction. Through modular design and session management, the system maintains reliability, scalability, and ease of deployment, demonstrating a significant improvement over traditional single-intent classification systems. Overall, the project validates the practicality of multi-intent recognition in enhancing customer support efficiency, reducing response times, and improving overall user satisfaction.The Flask-based implementation ensures a seamless user experience, providing real-time predictions, visualization of model performance, and analytics dashboards that track conversation metrics such as response time, intent distribution, and user satisfaction. Through modular design and session management, the system maintains reliability, scalability, and ease of deployment, demonstrating a significant improvement over traditional single-intent classification systems.

7.2 FUTURE ASPECTS

The system has significant potential for enhancement and expansion in multiple directions. One major improvement would be the integration of context-aware conversational AI, where the model not only detects intents but also maintains contextual understanding over long multi-turn conversations. Another future direction is the incorporation of advanced deep learning architectures, such as transformers or attention- based networks, to handle more complex user queries with better accuracy and generalization.

Additionally, the system could be extended to support voice-based interactions using speech-to-text and text-to-speech APIs, making it more versatile for real-time customer support. Implementing reinforcement learning techniques for adaptive response generation could further optimize the system by learning from user feedback and improving over time.

8. BIBLIOGRAPHY

8.BIBLIOGRAPHY

8.1 REFERENCES

[1] Huang Z, Liu H, Zhu J, Min J. Customer Sentiment Recognition in Conversation. Applied Sciences. 2023; 13(13):7807. https://doi.org/10.3390/app13137807

[2] Wagner N, Kraus M, Minker W, Griol D, Callejas Z. A Survey on Multi-User Conversational Interfaces. Applied Sciences. 2025; 15(13):7267. https://doi.org/10.3390/app15137267

[3] Lin C-C, Huang AYQ, Yang SJH. A Review of AI-Driven Conversational Chatbots . Sustainability. 2023; 15(5):4012. https://doi.org/10.3390/su15054012

[4] Bercaru G, Truică C-O, Chiru C-G, Rebedea T. Improving Intent Classification. 2023; 11(3):769. https://doi.org/10.3390/math11030769

[5] Chuang H-M, Cheng D-W. Conversational AI over Military Scenarios . Applied Sciences. 2022; 12(5):2494. https://doi.org/10.3390/app12052494

[6] Nicolescu L, Tudorache MT. Human-Computer Interaction in Customer Service: The Experience with AI Chatbots. Electronics. 2022; 11(10):1579. https://doi.org/10.3390/electronics11101579

[7] Merdivan E, Singh D, Hanke S, Kropf J, Holzinger A, Geist M. Human Annotated Dialogues Dataset for Natural Conversational Agents. Applied Sciences. 2020; 10(3):762. https://doi.org/10.3390/app10030762

[8] Izadi S, Forouzanfar M. Error Correction and Adaptation in Conversational AI: A Review of Techniques and Applications in Chatbots. AI. 2024; 5(2):803-841. https://doi.org/10.3390/ai5020041

[9] Al-Mutawa RF, Al-Aama AY. Arabic Opinion Classification of Customer Service Conversations Using Data Augmentation and Artificial Intelligence. Big Data and Cognitive Computing. 2024; 8(12):196. https://doi.org/10.3390/bdcc8120196

[10] Sultana T, Mandal AK, Saha H, Sultan MN, Hossain MD. Intent Identification by Semantically Analyzing the Search Query. Modelling. 2024; 5(1):292-314. https://doi.org/10.3390/modelling5010016

8.2 GITHUB LINK

https://github.com/Shivanigidya/Customer-support-dataset-with-multi-itent-annotations-for-conservational-AI/tree/main